

Backpropagation: Complete Derivations

Companion notes to Chapter 15, *Deep Learning Foundations*

Abstract

These notes derive backpropagation from the multivariable chain rule, with every intermediate step written out and every step explained rather than merely stated. The guiding claim is that backpropagation is not a separate algorithm bolted onto calculus but a single recursion — and that the same recursion, read a second time, is the reason deep networks were nearly untrainable for decades. Section 1 states the problem backpropagation solves and prices the naive alternative; Section 2 fixes notation and isolates the two structural facts that do all the work; Section 3 fixes mean squared error as a running example, computes the error signal at the last layer, takes one step down by hand, and only then generalises to the recursion, in index form and then in matrix form; Section 4 converts the error signal into parameter gradients and gives the mini-batch form; Section 5 works a complete numerical example by hand; Section 6 establishes the $O(P)$ versus $O(P^2)$ separation and discusses the memory it costs; Section 7 evaluates the output-layer base case for the two standard loss/activation pairings; Section 8 shows that the recursion that makes backpropagation cheap is what makes the gradient vanish, and derives the precise threshold; Sections 9 and 10 derive the two standard responses to that threshold, He initialization and the residual block. Section 11 describes how to verify an implementation.

Contents

1	The problem backpropagation solves	2
2	Setup and notation	2
2.1	Two structural facts	3
2.2	The central definition	4
3	The error-signal recursion	4
3.1	A concrete loss: mean squared error	4
3.2	Start at the last layer	5
3.3	One step down, by hand	5
3.4	The general inductive step	6
3.5	Matrix form and the unrolled product	8
4	From the error signal to parameter gradients	8
4.1	Mini-batch form	9
4.2	Shape check	9
4.3	The algorithm	10
5	A worked example	10
6	Cost, and what it buys	11
7	The output-layer base case, evaluated	12
7.1	Softmax with cross-entropy	12
7.2	Comparison with the mean-squared-error base case	13

8	Why the same recursion makes the gradient vanish	13
9	He initialization, from a variance constraint	14
10	The residual block	15
11	Verifying an implementation	17
12	Summary	17

1 The problem backpropagation solves

A network makes a prediction and it is wrong. Somewhere among millions of weights, some deserve more blame for that error than others, and training means adjusting each weight in proportion to its blame. The difficulty is that a weight buried three layers deep does not touch the loss directly: it perturbs a pre-activation, which perturbs an activation, which feeds every unit of the next layer, and so on until the effect finally reaches the output. Assigning blame through that tangle is the entire computational problem.

There is an obvious method. Nudge one weight by a small amount, run the network forward, see how much the loss moved, and divide:

$$\frac{\partial \mathcal{L}}{\partial \theta_i} \approx \frac{\mathcal{L}(\theta + \epsilon e_i) - \mathcal{L}(\theta - \epsilon e_i)}{2\epsilon}. \quad (1)$$

This is correct and requires no derivation at all. It is also unusable, for two independent reasons. First, cost: (1) gives one component of the gradient per pair of forward passes, so a network with P parameters needs $O(P)$ forward passes, each itself costing $O(P)$ work — $O(P^2)$ per training step. At $P = 10^8$ this is hopeless, and it is hopeless by a margin that no amount of hardware closes. Second, precision: (1) subtracts two nearly equal floating-point numbers, so it loses significant digits exactly when ϵ is small enough for the approximation to be accurate. There is a floor on how good the estimate can get, and it is not a very low floor.

Backpropagation computes the blame for *every* weight in a single backward sweep, at a cost of roughly one forward pass, and computes it exactly rather than approximately. It is the algorithm that makes deep learning computationally possible at all. The rest of these notes derive it and then examine what the derivation implies.

The organising idea, stated before the algebra so that the algebra has somewhere to land: if we can compute, for each layer, a single vector summarising “how much the loss changes when this layer’s pre-activation changes,” then (i) that vector immediately yields the gradients of that layer’s weights and biases, and (ii) the same vector for the layer below can be computed from it in one cheap step. Property (ii) is what makes the whole thing a recursion; property (i) is what makes the recursion worth running.

2 Setup and notation

Consider a feedforward network of L layers. Write $a^{(0)} = x \in \mathbb{R}^{n_0}$ for the input. For $l = 1, \dots, L$,

$$z^{(l)} = W^{(l)} a^{(l-1)} + b^{(l)}, \quad (2)$$

$$a^{(l)} = f(z^{(l)}), \quad (3)$$

where $W^{(l)} \in \mathbb{R}^{n_l \times n_{l-1}}$, $b^{(l)} \in \mathbb{R}^{n_l}$, and f is applied elementwise. A scalar loss $\mathcal{L}$ is computed from $a^{(L)}$ and the target y .

Equations (2)–(3), repeated L times, are the entire forward pass; nothing else happens. It is worth pausing on how little that is. Each layer performs one matrix–vector product, one vector addition, and one elementwise function application. The apparent complexity of a deep network is entirely a matter of *repetition*, not of any individual operation being complicated — and this is precisely why a single recursion can differentiate the whole thing.

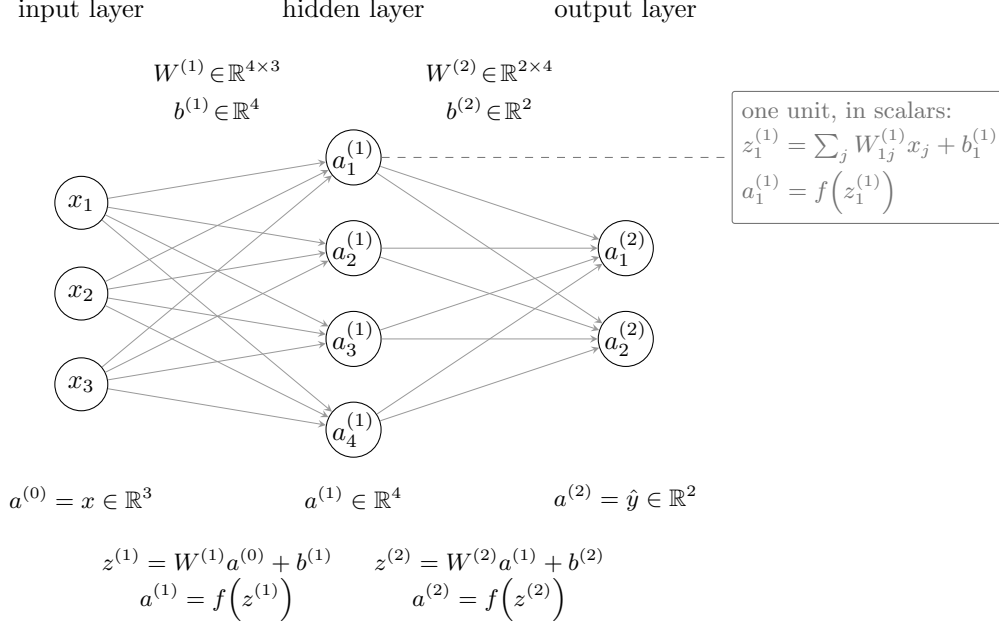


Figure 1: Forward pass of a $3 \rightarrow 4 \rightarrow 2$ network, labelled to match Eqs. (2)–(3). Every edge carries one entry of $W^{(l)}$; every circle sums its incoming edges, adds the bias, and applies f elementwise. The callout expands one unit into the scalar form used in the index derivations of Section 3. Each $z^{(l)}$ and $a^{(l)}$ produced here is cached, because the backward pass of Figure 2 consumes both.

Some vocabulary, since the two symbols z and a are easy to conflate and the derivation depends on keeping them apart. The vector $z^{(l)}$ is the *pre-activation*: the raw weighted sum, before the nonlinearity. The vector $a^{(l)}$ is the *activation*: what the layer actually sends onward. The distinction matters because $W^{(l)}$ and $b^{(l)}$ influence the loss only through $z^{(l)}$, whereas $z^{(l)}$ influences the loss only through $a^{(l)}$. Each variable is a genuine bottleneck, and the derivation exploits both.

2.1 Two structural facts

Almost everything below rests on two properties of the architecture. Neither is deep, but both are load-bearing, and it is useful to see them isolated before they are used.

- (i) **The activation Jacobian is diagonal.** Because f is applied elementwise, unit j ’s pre-activation affects unit j ’s activation and no other. So

$$\frac{\partial a_i^{(l)}}{\partial z_j^{(l)}} = f'(z_j^{(l)}) \delta_{ij}, \quad \text{i.e.} \quad \frac{\partial a^{(l)}}{\partial z^{(l)}} = D^{(l)} := \text{diag}\left(f'(z^{(l)})\right). \quad (4)$$

In general a Jacobian between two n -vectors has n^2 entries; here all but n of them vanish. This is why the activation contributes $O(n)$ rather than $O(n^2)$ work to the backward pass, and it is why the final formulas contain a Hadamard product instead of a matrix multiplication.

- (ii) **Diagonal multiplication is the Hadamard product.** For diagonal D with entries d_i , $(Dv)_i = d_i v_i$, which is exactly the elementwise product $D^{(l)}v = f'(z^{(l)}) \odot v$. Nothing is being approximated when $\odot$ replaces a Jacobian in what follows; the two are the same operation written two ways, and the elementwise form is the one an implementation actually uses because it never materialises the $n \times n$ matrix.

2.2 The central definition

Definition. The *error signal* at layer l is

$$\delta^{(l)} := \frac{\partial \mathcal{L}}{\partial z^{(l)}} \in \mathbb{R}^{n_l}, \quad \delta_j^{(l)} = \frac{\partial \mathcal{L}}{\partial z_j^{(l)}}. \quad (5)$$

Read $\delta_j^{(l)}$ as: *if unit j of layer l had produced a slightly larger pre-activation, by how much would the final loss have changed?* A large positive entry says that unit contributed to the error by firing too strongly; a large negative entry says it should have fired more strongly; an entry near zero says the unit was, for this example, irrelevant. It is a per-unit blame score, and it is the object the entire algorithm transports backward.

Why differentiate with respect to z rather than with respect to a , or directly with respect to W ? Because $z^{(l)}$ is the unique bottleneck through which layer l 's parameters reach the loss. Every one of the $n_l \times n_{l-1}$ weights in $W^{(l)}$ influences $\mathcal{L}$ only by changing some entry of $z^{(l)}$. So once the n_l numbers in $\delta^{(l)}$ are known, all $n_l \times n_{l-1}$ weight derivatives follow by one multiplication each — no further chain-rule work is required. Choosing $a^{(l)}$ instead would leave the activation derivative still to be applied at every use; choosing $W^{(l)}$ directly would forfeit the sharing entirely, since there would be nothing left to pass to the layer below. The choice of z is what compresses all downstream dependence into one vector per layer, and that compression *is* the algorithm.

3 The error-signal recursion

Everything in this section is organised around one instruction: *always start from the last layer*. That is not a stylistic preference. The loss is a function of the network's output and nothing else, so the output layer is the only place where a derivative can be written down without knowing anything about the layers below it. Every other layer's blame is defined in terms of the layer above, so there is no way to begin anywhere else — and once the last layer is known, each step downward is mechanical. The plan is therefore: fix a concrete loss, compute $\delta^{(L)}$ from it, compute $\delta^{(L-1)}$ from $\delta^{(L)}$ by hand, and only then generalise the second computation into a recursion.

3.1 A concrete loss: mean squared error

A general $\mathcal{L}$ makes the derivation harder to follow than it needs to be, so fix the simplest one. For a network with n_L outputs and target $y \in \mathbb{R}^{n_L}$, the squared error on one example is

$$\mathcal{L} = \frac{1}{2} \|a^{(L)} - y\|^2 = \frac{1}{2} \sum_{i=1}^{n_L} \left(a_i^{(L)} - y_i\right)^2. \quad (6)$$

The $\frac{1}{2}$ is there purely so that the 2 from the power rule cancels; it changes nothing but the tidiness of what follows. Over a batch of m examples one averages, giving the *mean* squared error, but since differentiation is linear the per-example derivation below is the whole story and the batch case only adds a $1/m$.

The one derivative we need from (6) is with respect to the network's output. Differentiating the sum, only the $i = j$ term contains $a_j^{(L)}$:

$$\frac{\partial \mathcal{L}}{\partial a_j^{(L)}} = \frac{1}{2} \cdot 2 \left(a_j^{(L)} - y_j \right) = a_j^{(L)} - y_j, \quad \text{i.e.} \quad \nabla_{a^{(L)}} \mathcal{L} = a^{(L)} - y. \quad (7)$$

Prediction minus target. That is the entire loss-specific content of the derivation: every subsequent formula in these notes uses the loss only through (7), so swapping in a different loss changes this one vector and nothing else.

3.2 Start at the last layer

Now compute $\delta^{(L)} = \partial \mathcal{L} / \partial z^{(L)}$. The loss depends on $z^{(L)}$ only through $a^{(L)} = f(z^{(L)})$, so the chain rule gives, using the diagonal Jacobian (4),

$$\delta_j^{(L)} = \sum_i \frac{\partial \mathcal{L}}{\partial a_i^{(L)}} \frac{\partial a_i^{(L)}}{\partial z_j^{(L)}} = \frac{\partial \mathcal{L}}{\partial a_j^{(L)}} f'(z_j^{(L)}), \quad \text{i.e.} \quad \delta^{(L)} = \nabla_{a^{(L)}} \mathcal{L} \odot f'(z^{(L)}). \quad (8)$$

The sum over i collapses to the single term $i = j$ because the Jacobian is diagonal by fact (i). Substituting the mean-squared-error gradient (7),

$$\boxed{\delta^{(L)} = \left(a^{(L)} - y \right) \odot f'(z^{(L)})} \quad \text{or, per unit,} \quad \delta_j^{(L)} = \left(a_j^{(L)} - y_j \right) f'(z_j^{(L)}). \quad (9)$$

Read it as a product of two intuitive quantities. The first, $a_j^{(L)} - y_j$, is *how wrong output j was, and in which direction* — positive if the network overshot, negative if it undershot. The second, $f'(z_j^{(L)})$, is *how responsive that output still is*: if the unit is sitting on a flat part of its activation curve, then changing $z_j^{(L)}$ barely changes $a_j^{(L)}$, so the pre-activation deserves less blame even though the prediction was wrong. Both factors come from numbers the forward pass already computed, so the base case costs essentially nothing.

Two special cases worth having at hand. If the output layer is *linear*, which is the usual choice for regression with MSE, then $f' \equiv 1$ and

$$\delta^{(L)} = a^{(L)} - y \quad (10)$$

exactly — the error itself, with no damping. If instead the output is a sigmoid, then $f'(z) = a(1-a) \leq \frac{1}{4}$, so $\delta_j^{(L)} = (a_j^{(L)} - y_j) a_j^{(L)} (1 - a_j^{(L)})$, and notice the pathology: a unit that predicted 0.99 when the target was 0 is maximally wrong, yet $f'(0.99) \approx 0.01$ crushes its blame to near zero. Learning is slowest exactly where it is most needed. Section 7 shows that swapping MSE for cross-entropy removes this factor entirely, which is the real reason for the standard pairing.

3.3 One step down, by hand

With $\delta^{(L)}$ in hand, take a single step backward to layer $L - 1$ concretely, before generalising. The question is: how does $z_j^{(L-1)}$ — one pre-activation in the second-to-last layer — influence the loss?

Trace the path forward. First $z_j^{(L-1)}$ becomes an activation, $a_j^{(L-1)} = f(z_j^{(L-1)})$. That activation is then sent to *every* unit of the output layer, since

$$z_k^{(L)} = \sum_i W_{ki}^{(L)} a_i^{(L-1)} + b_k^{(L)} \quad \text{for } k = 1, \dots, n_L. \quad (11)$$

So $z_j^{(L-1)}$ reaches the loss along n_L separate routes, one per output unit, and the multivariable chain rule says to add up what each route contributes:

$$\delta_j^{(L-1)} = \frac{\partial \mathcal{L}}{\partial z_j^{(L-1)}} = \sum_{k=1}^{n_L} \underbrace{\frac{\partial \mathcal{L}}{\partial z_k^{(L)}}}_{\delta_k^{(L)}} \cdot \frac{\partial z_k^{(L)}}{\partial z_j^{(L-1)}}. \quad (12)$$

The first factor is already known — it is (9), computed a moment ago. For the second, differentiate the display above: $z_k^{(L)}$ depends on $z_j^{(L-1)}$ only through the $i = j$ term, because f is elementwise, so $\partial z_k^{(L)} / \partial z_j^{(L-1)} = W_{kj}^{(L)} f'(z_j^{(L-1)})$. Substituting and pulling the common factor out of the sum,

$$\delta_j^{(L-1)} = f'(z_j^{(L-1)}) \sum_{k=1}^{n_L} W_{kj}^{(L)} \delta_k^{(L)}, \quad \text{i.e.} \quad \delta^{(L-1)} = \left(\left(W^{(L)} \right)^\top \delta^{(L)} \right) \odot f'(z^{(L-1)}). \quad (13)$$

In words: *collect the blame from every unit this one feeds, weighted by the strength of the connection, then scale by the local slope.* Note that the loss has now disappeared from the calculation entirely — once $\delta^{(L)}$ was computed in Section 3.2, everything below depends on the loss only through that vector. This is why a single implementation of the backward pass serves every loss function.

Notice also what did *not* happen: nothing in this step referred to L specifically. The only inputs were the layer’s own z , the weights above it, and the δ above it. That is the observation the next subsection formalises — the step from L to $L - 1$ is already the general step.

3.4 The general inductive step

Section 3.3 took one step, from L to $L - 1$. Repeat it: to get $\delta^{(L-2)}$ we would ask how $z_j^{(L-2)}$ reaches the loss, find that it does so only through the units of layer $L - 1$, and run the identical argument with $\delta^{(L-1)}$ in place of $\delta^{(L)}$. Nothing about the argument used the value of the layer index, so it holds at every depth. The following proposition states that in general; the proof is Eq. (13)’s derivation written with l in place of $L - 1$, and with each step labelled.

Proposition 3.1 (Backward recursion). *For $l = 1, \dots, L - 1$,*

$$\delta^{(l)} = \left(\left(W^{(l+1)} \right)^\top \delta^{(l+1)} \right) \odot f'(z^{(l)}). \quad (14)$$

Proof. We proceed in four steps. Step 1 sets up the chain rule and is where the entire content lies; Steps 2–4 are bookkeeping that evaluate what Step 1 leaves open.

Step 1 — identify the paths. The coordinate $z_j^{(l)}$ influences $\mathcal{L}$ only by way of the next layer’s pre-activations $z_1^{(l+1)}, \dots, z_{n_{l+1}}^{(l+1)}$: it first becomes $a_j^{(l)}$, and $a_j^{(l)}$ is then fanned out to every unit of layer $l + 1$. Nothing downstream of layer $l + 1$ sees $z^{(l)}$ except through those n_{l+1} intermediaries. This is exactly the situation the multivariable chain rule addresses — one variable influencing a scalar through several intermediate variables — and its prescription is to sum the contributions of all such paths:

$$\frac{\partial \mathcal{L}}{\partial z_j^{(l)}} = \sum_{k=1}^{n_{l+1}} \frac{\partial \mathcal{L}}{\partial z_k^{(l+1)}} \frac{\partial z_k^{(l+1)}}{\partial z_j^{(l)}} = \sum_{k=1}^{n_{l+1}} \delta_k^{(l+1)} \frac{\partial z_k^{(l+1)}}{\partial z_j^{(l)}}. \quad (15)$$

The second equality is just the definition (5) applied one layer up, and it is the moment the recursion appears: the unknown at layer l has been expressed in terms of the already-known quantity at layer $l + 1$. Everything after this is evaluation.

Step 2 — evaluate the inner partial derivative. Write layer $l + 1$'s pre-activation explicitly in terms of layer l 's:

$$z_k^{(l+1)} = \sum_{i=1}^{n_l} W_{ki}^{(l+1)} f(z_i^{(l)}) + b_k^{(l+1)}. \quad (16)$$

Differentiating with respect to $z_j^{(l)}$, only the $i = j$ term survives — because f is elementwise, unit i 's pre-activation does not touch unit j 's activation for $i \neq j$ (fact (i) again). The bias contributes nothing, being constant in $z^{(l)}$. Hence

$$\frac{\partial z_k^{(l+1)}}{\partial z_j^{(l)}} = W_{kj}^{(l+1)} f'(z_j^{(l)}). \quad (17)$$

Read this as the two things that happen along one backward hop: the signal is scaled by the weight on the edge it travels, and by the local slope of the activation it passes through.

Step 3 — substitute and factor. Inserting (17) into (15):

$$\delta_j^{(l)} = \sum_{k=1}^{n_{l+1}} \delta_k^{(l+1)} W_{kj}^{(l+1)} f'(z_j^{(l)}) = f'(z_j^{(l)}) \sum_{k=1}^{n_{l+1}} W_{kj}^{(l+1)} \delta_k^{(l+1)}. \quad (18)$$

Pulling $f'(z_j^{(l)})$ out of the sum is legitimate because it carries the index j , not the summation index k : it is the same constant in every term. This is a small step but a consequential one, since it separates the two factors of the hop — a weight part that mixes across units, and an activation part that acts on each unit independently.

Step 4 — recognise the matrix–vector product. The remaining sum contracts $\delta^{(l+1)}$ against the *first* index of $W^{(l+1)}$. Matrix–vector multiplication contracts against the second index, so what appears here is the transpose:

$$\sum_k W_{kj}^{(l+1)} \delta_k^{(l+1)} = \sum_k \left(W^{(l+1)} \right)_{jk}^\top \delta_k^{(l+1)} = \left[\left(W^{(l+1)} \right)^\top \delta^{(l+1)} \right]_j. \quad (19)$$

Collecting (18) over all j , and using fact (ii) to write the elementwise multiplication by f' as $\odot$, gives (14). $\square$

Remark 3.2 (Why the transpose is not a convention). The transpose is forced by shape, not chosen for tidiness. Forward, $W^{(l+1)}$ maps $\mathbb{R}^{n_l} \rightarrow \mathbb{R}^{n_{l+1}}$; backward, the signal must travel from $\mathbb{R}^{n_{l+1}}$ to $\mathbb{R}^{n_l}$, and the only linear map built from $W^{(l+1)}$ with those dimensions is its transpose. Checking shapes: $(n_l \times n_{l+1})(n_{l+1} \times 1) = n_l \times 1$, then $\odot$ with $f'(z^{(l)}) \in \mathbb{R}^{n_l}$ leaves $\mathbb{R}^{n_l}$, which is what $\delta^{(l)}$ must be. A useful reading is that the forward pass distributes each unit's activation to many downstream units, and the backward pass collects each unit's blame back from those same many units — fan-out becomes fan-in, and transposition is what fan-out-becomes-fan-in looks like in matrix notation.

Remark 3.3 (The two factors). Read (14) once more and count the operations that touch δ on the way down: multiplication by $(W^{(l+1)})^\top$, and multiplication by $f'(z^{(l)})$. Two factors, applied *multiplicatively*, once per layer, with no additive term anywhere. Nothing in the derivation guarantees those factors are near 1; they are whatever the weights and the activation happen to make them. Section 8 takes this observation seriously, and it turns out to explain most of the architectural history of deep learning.

3.5 Matrix form and the unrolled product

Using $D^{(l)} = \text{diag}(f'(z^{(l)}))$ from (4), the recursion is a pure sequence of linear maps:

$$\delta^{(l)} = D^{(l)} \left(W^{(l+1)} \right)^\top \delta^{(l+1)}. \quad (20)$$

This form obscures the efficiency (one would never build $D^{(l)}$ explicitly) but exposes the structure, because now the whole backward pass is visibly a product of matrices. Unrolling from the output down to layer l :

$$\delta^{(l)} = \left(\prod_{k=l+1}^L D^{(k-1)} \left(W^{(k)} \right)^\top \right) \delta^{(L)}, \quad (21)$$

with the factors ordered so that the $k = l + 1$ term is applied last. Equation (21) is the single most consequential line in these notes: the blame reaching layer l is the output blame passed through a product of $2(L - l)$ matrices. It is used as stated in Sections 8 and 10.

4 From the error signal to parameter gradients

The recursion transports blame, but blame is not yet what an optimizer consumes. This section converts $\delta^{(l)}$ into the gradients of the actual parameters — and the conversion is where the payoff for defining δ with respect to z arrives.

Proposition 4.1. *For every l ,*

$$\frac{\partial \mathcal{L}}{\partial W^{(l)}} = \delta^{(l)} \left(a^{(l-1)} \right)^\top, \quad \frac{\partial \mathcal{L}}{\partial b^{(l)}} = \delta^{(l)}. \quad (22)$$

Proof. From $z_i^{(l)} = \sum_j W_{ij}^{(l)} a_j^{(l-1)} + b_i^{(l)}$, observe that the weight $W_{ij}^{(l)}$ appears in exactly one pre-activation, namely $z_i^{(l)}$ — it is the coefficient on the edge running from unit j below to unit i here, and that edge feeds one destination. So the chain-rule sum over all pre-activations has a single surviving term:

$$\frac{\partial \mathcal{L}}{\partial W_{ij}^{(l)}} = \sum_m \frac{\partial \mathcal{L}}{\partial z_m^{(l)}} \frac{\partial z_m^{(l)}}{\partial W_{ij}^{(l)}} = \delta_i^{(l)} \frac{\partial z_i^{(l)}}{\partial W_{ij}^{(l)}} = \delta_i^{(l)} a_j^{(l-1)}, \quad (23)$$

which is the (i, j) entry of the outer product $\delta^{(l)} (a^{(l-1)})^\top$. Identically, $\partial z_i^{(l)} / \partial b_i^{(l)} = 1$ gives $\partial \mathcal{L} / \partial b_i^{(l)} = \delta_i^{(l)}$. $\square$

Equation (23) deserves reading in words, because it is the most interpretable formula in the subject: *the gradient of a weight is the blame at its destination times the activation at its source*. A weight is adjusted a lot when the unit it feeds was very wrong *and* the unit it reads from was very active. If either end was quiet — the destination blameless, or the source silent — that weight barely moves. This is also why dead units are self-perpetuating: a unit whose activation is identically zero contributes $a_j^{(l-1)} = 0$ to every outgoing weight gradient, so none of those weights can change.

The outer-product form is also what hardware executes: one rank-one update per example, or a single matrix product over a mini-batch.

4.1 Mini-batch form

Real training processes many examples at once, and the change is smaller than it looks. Let $A^{(l-1)} \in \mathbb{R}^{n_{l-1} \times m}$ and $\Delta^{(l)} \in \mathbb{R}^{n_l \times m}$ stack m examples column-wise, and let $\mathcal{L}$ be the *sum* over the batch. Because the loss is a sum over examples and each example's forward pass is independent, the gradient is the sum of per-example gradients; summing (22) over examples turns m outer products into a single matrix product:

$$\frac{\partial \mathcal{L}}{\partial W^{(l)}} = \Delta^{(l)} \left(A^{(l-1)} \right)^\top \in \mathbb{R}^{n_l \times n_{l-1}}, \quad \frac{\partial \mathcal{L}}{\partial b^{(l)}} = \Delta^{(l)} \mathbf{1}_m, \quad (24)$$

where $\mathbf{1}_m$ is the all-ones vector, so the bias gradient is a row-sum. The recursion (14) itself applies column-wise, unchanged: $\Delta^{(l)} = ((W^{(l+1)})^\top \Delta^{(l+1)}) \odot f'(Z^{(l)})$. For a mean loss rather than a sum, divide by m . The practical significance is that batching converts matrix–vector products into matrix–matrix products, which is what makes a GPU worth using.

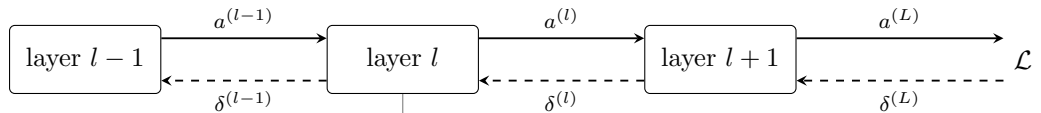
4.2 Shape check

Every quantity in the backward pass has a shape determined by the forward pass, and a shape mismatch is the most common implementation bug. For reference:

Quantity	Shape (single example)	Shape (batch of m)
$a^{(l)}, z^{(l)}, \delta^{(l)}, \partial \mathcal{L} / \partial b^{(l)}$	$n_l \times 1$	$n_l \times m$ (except b : $n_l \times 1$)
$W^{(l)}, \partial \mathcal{L} / \partial W^{(l)}$	$n_l \times n_{l-1}$	$n_l \times n_{l-1}$
$(W^{(l+1)})^\top \delta^{(l+1)}$	$n_l \times 1$	$n_l \times m$

Two invariants follow and are worth asserting in code: a parameter gradient always has exactly the shape of the parameter itself, and $\delta^{(l)}$ always has the shape of $z^{(l)}$. Neither ever depends on the batch size in the case of the parameters, because the batch dimension is summed out by the matrix product.

forward sweep: cache every $z^{(l)}$ and $a^{(l)}$



backward sweep: one $\delta^{(l)}$ per layer, computed once

on receiving $\delta^{(l)}$, layer l emits

$$\frac{\partial \mathcal{L}}{\partial W^{(l)}} = \delta^{(l)} (a^{(l-1)})^\top, \quad \frac{\partial \mathcal{L}}{\partial b^{(l)}} = \delta^{(l)}$$

and passes down $\delta^{(l-1)} = ((W^{(l)})^\top \delta^{(l)}) \odot f'(z^{(l-1)})$

Figure 2: The two sweeps. Forward (solid), every z and a is cached; backward (dashed), a single δ per layer is computed once and then reused by *both* the weight gradient of that layer and the next δ down. That reuse — not the chain rule itself — is what makes backpropagation cost $O(P)$ rather than $O(P^2)$. Note that the dashed arrows carry the multiplication by $(W^{(l)})^\top$ and $f'(z^{(l-1)})$ analysed in Section 8: the gradient is scaled once per hop.

4.3 The algorithm

Forward	for $l = 1, \dots, L$: $z^{(l)} \leftarrow W^{(l)}a^{(l-1)} + b^{(l)}$, $a^{(l)} \leftarrow f(z^{(l)})$; cache $z^{(l)}, a^{(l)}$
Base	$\delta^{(L)} \leftarrow \nabla_{a^{(L)}} \mathcal{L} \odot f'(z^{(L)})$
Backward	for $l = L, \dots, 1$: emit $\delta^{(l)}(a^{(l-1)})^\top$ and $\delta^{(l)}$; $\delta^{(l-1)} \leftarrow ((W^{(l)})^\top \delta^{(l)}) \odot f'(z^{(l-1)})$

The essential point is contained in the ordering of the backward loop. When the algorithm reaches layer l , it does *not* recompute how layer l influences the loss from scratch; it consumes the $\delta^{(l+1)}$ that the previous iteration already produced, and uses it twice — once for this layer’s parameter gradients, once to produce $\delta^{(l)}$. That reuse of a subproblem’s solution is precisely dynamic programming, and backpropagation is best described as memoization over the network’s computation graph. The subproblems are the δ ’s; the overlapping structure is that every weight in layer l needs the same $\delta^{(l)}$; and the memo table is the list of cached δ ’s. Naming this correspondence is not decoration — it is the reason the cost is $O(P)$ rather than $O(P^2)$, as Section 6 makes explicit.

Backpropagation is also one instance of a more general technique, *reverse-mode automatic differentiation*. Reverse mode computes the gradient of one output with respect to many inputs in a single sweep; forward mode does the opposite, one input with respect to many outputs. Deep learning has exactly one scalar output (the loss) and millions of inputs (the weights), so reverse mode wins by a factor of P . That asymmetry — and not any property peculiar to neural networks — is why the field differentiates backward.

5 A worked example

Abstract index manipulation is easy to follow and easy to not quite believe. This section runs the entire algorithm on a network small enough to check by hand, following exactly the route of Section 3: mean squared error, start at the last layer, then step down. Every formula used here has already been derived; only the numbers are new.

Take a $2 \rightarrow 2 \rightarrow 1$ network with $f = \tanh$ in the hidden layer, a linear output, and squared-error loss $\mathcal{L} = \frac{1}{2}(a^{(2)} - y)^2$. Let

$$W^{(1)} = \begin{pmatrix} 1 & -1 \\ 0 & 2 \end{pmatrix}, \quad b^{(1)} = \begin{pmatrix} 0 \\ 0 \end{pmatrix}, \quad W^{(2)} = \begin{pmatrix} 1 & 1 \end{pmatrix}, \quad b^{(2)} = 0, \quad x = \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \quad y = 0. \quad (25)$$

Forward pass.

$$z^{(1)} = W^{(1)}x + b^{(1)} = \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \quad a^{(1)} = \tanh(z^{(1)}) = \begin{pmatrix} 0.7616 \\ 0 \end{pmatrix}, \quad (26)$$

$$z^{(2)} = W^{(2)}a^{(1)} + b^{(2)} = 0.7616, \quad a^{(2)} = z^{(2)} = 0.7616. \quad (27)$$

The loss is $\mathcal{L} = \frac{1}{2}(0.7616 - 0)^2 = 0.29001$.

Base case. With a linear output, $f' \equiv 1$, so (8) gives

$$\delta^{(2)} = (a^{(2)} - y) \cdot 1 = 0.7616. \quad (28)$$

Output-layer gradients. By (22),

$$\frac{\partial \mathcal{L}}{\partial W^{(2)}} = \delta^{(2)} (a^{(1)})^\top = 0.7616 \cdot (0.7616 \ 0) = (0.5800 \ 0), \quad \frac{\partial \mathcal{L}}{\partial b^{(2)}} = 0.7616. \quad (29)$$

Note the second entry: the gradient on the weight from hidden unit 2 is exactly zero, because that unit’s activation was zero. This is the “blame at destination times activation at source” reading of (23), visible in a single number.

Propagate down. Since $\tanh'(z) = 1 - \tanh^2(z)$, we have $f'(z^{(1)}) = (1 - 0.7616^2, 1 - 0)^T = (0.4200, 1)^T$. Then by (14),

$$\delta^{(1)} = \left(\left(W^{(2)} \right)^T \delta^{(2)} \right) \odot f'(z^{(1)}) = \begin{pmatrix} 0.7616 \\ 0.7616 \end{pmatrix} \odot \begin{pmatrix} 0.4200 \\ 1 \end{pmatrix} = \begin{pmatrix} 0.3198 \\ 0.7616 \end{pmatrix}. \quad (30)$$

Hidden unit 2 receives substantial blame even though its outgoing weight gradient was zero — the two are different questions. Its activation was zero, so it could not have influenced the loss through its *current* outgoing weight in a way that depends on that weight’s value; but changing its pre-activation would certainly change the output, which is what δ measures.

Hidden-layer gradients.

$$\frac{\partial \mathcal{L}}{\partial W^{(1)}} = \delta^{(1)} x^\top = \begin{pmatrix} 0.3198 \\ 0.7616 \end{pmatrix} \begin{pmatrix} 1 & 0 \end{pmatrix} = \begin{pmatrix} 0.3198 & 0 \\ 0.7616 & 0 \end{pmatrix}, \quad \frac{\partial \mathcal{L}}{\partial b^{(1)}} = \begin{pmatrix} 0.3198 \\ 0.7616 \end{pmatrix}. \quad (31)$$

The entire second column is zero because $x_2 = 0$: weights reading from an inactive input receive no gradient, for the same reason as before.

Verification. Check one entry by finite differences. Perturbing $W_{11}^{(1)}$ by $\epsilon = 10^{-5}$ and recomputing the forward pass gives $\mathcal{L}(+\epsilon) = 0.2900160$ and $\mathcal{L}(-\epsilon) = 0.2900096$, whence $(\mathcal{L}(+\epsilon) - \mathcal{L}(-\epsilon))/2\epsilon \approx 0.31985$, matching the analytic value to five significant figures. Section 11 generalises this check.

6 Cost, and what it buys

Let P denote the number of parameters.

	Cost	Source
Forward pass	$O(P)$	one multiply-add per weight
Backward pass	$O(P)$, $\approx 2 \times$ forward	each layer computes both $\partial \mathcal{L} / \partial W^{(l)}$ and $\delta^{(l-1)}$
Finite differences	$O(P)$ forward passes, i.e. $O(P^2)$	one perturbation per parameter

The factor of roughly two in the backward pass is not an empirical fudge: by the algorithm table, each layer performs two matrix products where the forward pass performed one — the outer product $\delta^{(l)} (a^{(l-1)})^\top$ for the parameter gradient, and the product $(W^{(l)})^\top \delta^{(l)}$ to continue the recursion. Both have the same cost as the forward product $W^{(l)} a^{(l-1)}$, hence $\approx 2 \times$.

The comparison with (1) is the whole justification for the derivation. Finite differences perturbs each parameter separately and therefore rediscovers, P times over, the same intermediate quantities. Backpropagation exploits exactly the structure the naive method discards: by Proposition 3.1, $\delta^{(l)}$ is built from $\delta^{(l+1)}$, so each intermediate is computed once and then reused by every weight in that layer. The whole field lives in the gap between $O(P)$ and $O(P^2)$; at $P = 1.5 \times 10^5$ — a small model by any modern standard — the gap is already about five orders of magnitude per training step.

The price: activation memory. Nothing is free, and here the cost is memory. Look again at the backward step: computing $\partial \mathcal{L} / \partial W^{(l)}$ requires $a^{(l-1)}$, and computing $\delta^{(l-1)}$ requires $z^{(l-1)}$. Both were produced during the forward pass, long before they are needed, so both must be *kept*. Over L layers and a batch of m examples this is $O(Lm \cdot \text{width})$ numbers, and for large models it is usually activation memory, not parameter memory, that exhausts the device.

The standard remedy is *gradient checkpointing* (activation recomputation): store only a sparse subset of activations on the forward pass and recompute the rest on demand during the backward pass. With checkpoints every $\sqrt{L}$ layers, activation memory falls from $O(L)$ to $O(\sqrt{L})$ at the cost of roughly one extra forward pass. It is worth noticing that this trade — recompute rather than store — is the exact mirror of the memoization that makes backpropagation fast in the first place. The algorithm stores to avoid recomputing; checkpointing recomputes to avoid storing; and which is preferable depends entirely on whether time or memory is the binding constraint.

Where finite differences still belongs. Not in training, but in testing: gradient-checking a hand-written backward pass on a small model, as in Section 11. Its $O(P^2)$ cost is irrelevant when P is a few hundred, and its independence from the code being tested is exactly what makes it a trustworthy oracle.

7 The output-layer base case, evaluated

Equation (8) was stated for general f and $\mathcal{L}$. In practice the output activation and the loss are chosen together, and for the two standard pairings the base case collapses to something far simpler than the general formula suggests. The collapse is not a coincidence, and understanding why it happens explains a standard piece of practical advice.

7.1 Softmax with cross-entropy

Let $z := z^{(L)} \in \mathbb{R}^C$, $\hat{y}_i = e^{z_i} / \sum_c e^{z_c}$, and $\mathcal{L} = -\sum_i y_i \log \hat{y}_i$ with y one-hot. Softmax is not elementwise — every output depends on every input, through the shared denominator — so fact (i) does not apply and the full Jacobian is needed.

Lemma 7.1 (Softmax Jacobian). $\frac{\partial \hat{y}_i}{\partial z_j} = \hat{y}_i(\delta_{ij} - \hat{y}_j)$.

Proof. Write $S = \sum_c e^{z_c}$, so $\hat{y}_i = e^{z_i} / S$ and $\partial S / \partial z_j = e^{z_j}$. The numerator e^{z_i} depends on z_j only when $i = j$, contributing the δ_{ij} ; the denominator depends on z_j always, contributing the second term. By the quotient rule,

$$\frac{\partial \hat{y}_i}{\partial z_j} = \frac{\delta_{ij} e^{z_i} S - e^{z_i} e^{z_j}}{S^2} = \delta_{ij} \hat{y}_i - \hat{y}_i \hat{y}_j. \quad (32)$$

□

Corollary 7.2. $\delta^{(L)} = \hat{y} - y$.

Proof. Since $\partial \mathcal{L} / \partial \hat{y}_i = -y_i / \hat{y}_i$,

$$\delta_j^{(L)} = \sum_i \left(-\frac{y_i}{\hat{y}_i} \right) \hat{y}_i (\delta_{ij} - \hat{y}_j) = -\sum_i y_i (\delta_{ij} - \hat{y}_j) = -y_j + \hat{y}_j \sum_i y_i = \hat{y}_j - y_j, \quad (33)$$

where the $\hat{y}_i$ from the Jacobian cancels the $1/\hat{y}_i$ from the log derivative, and the last step uses $\sum_i y_i = 1$. □

The result is worth dwelling on: the error signal at the output is simply *prediction minus target*, with no activation derivative surviving at all. The saturating factor cancelled exactly against the $1/\hat{y}$ produced by differentiating the logarithm. That cancellation is why cross-entropy is paired with softmax rather than squared error being used for classification. With squared error the base case retains a factor $\sigma'(z) \leq \frac{1}{4}$, which is near zero precisely when the network is confidently wrong — so the gradient, and hence the learning rate on that example, is

smallest exactly where the model most needs correcting. Cross-entropy removes that pathology by construction.

A practical corollary: implementations fuse softmax and cross-entropy into one operation (`softmax_cross_entropy_with_logits` and its equivalents) rather than composing them. The fused version computes $\hat{y} - y$ directly and never forms the intermediate Jacobian, which is both faster and numerically better behaved, since it avoids taking the logarithm of a possibly tiny $\hat{y}$.

7.2 Comparison with the mean-squared-error base case

It is worth putting the two side by side, since they are the two base cases a practitioner ever writes. Section 3.2 derived, for MSE, $\delta^{(L)} = (a^{(L)} - y) \odot f'(z^{(L)})$, which becomes exactly $a^{(L)} - y$ when the output layer is linear — the standard regression setup. The corollary above gives $\delta^{(L)} = \hat{y} - y$ for softmax with cross-entropy — the standard classification setup.

The two agree in form, but they arrive there by different routes, and the difference matters when the pairing is broken. In regression the f' factor is absent because the output activation is the identity, so there was never anything to damp the signal. In classification the f' factor was present and nontrivial, and it *cancelled* against the loss derivative. Pair softmax with MSE instead, and no cancellation occurs: the base case retains a factor bounded by $\frac{1}{4}$ that shrinks toward zero on confident mistakes, exactly the pathology flagged after Eq. (9). The lesson is that loss and output activation are chosen jointly, and $\delta^{(L)}$ is where one sees whether the choice was a good one.

8 Why the same recursion makes the gradient vanish

Remark 3.3 observed that each backward step multiplies δ by two factors, and (21) showed the blame reaching layer l is a product of $2(L - l)$ matrices. This section takes that seriously. The intuition is elementary — multiply many numbers together and, if they are consistently below one, the product shrinks exponentially — and the following proposition makes the threshold precise.

Proposition 8.1 (Vanishing/exploding condition). *Let $\gamma := \sup_z |f'(z)|$ and let $\sigma_{\max}(W)$ denote the largest singular value of W . Then for the unrolled recursion (21),*

$$\|\delta^{(l)}\| \leq (\gamma \cdot \sigma_{\max})^{L-l} \|\delta^{(L)}\| \quad (34)$$

whenever every layer satisfies $\sigma_{\max}(W^{(k)}) \leq \sigma_{\max}$.

Proof. We proceed in three steps.

Step 1 — submultiplicativity. The operator norm satisfies $\|AB\| \leq \|A\|\|B\|$, since the largest amount AB can stretch a vector is at most the largest stretch of B followed by the largest stretch of A . Applying this repeatedly to the product in (21),

$$\|\delta^{(l)}\| \leq \left(\prod_{k=l+1}^L \|D^{(k-1)}\| \cdot \left\| \left(W^{(k)} \right)^\top \right\| \right) \|\delta^{(L)}\|. \quad (35)$$

Operator norms of transposes are equal, so the transpose may be dropped from here on.

Step 2 — bound each factor. $D^{(k)}$ is diagonal, so it stretches most along whichever axis has the largest entry, and its operator norm is that entry's absolute value — which is bounded by the activation's steepest slope anywhere: $\|D^{(k)}\| \leq \gamma$. For a general matrix the operator norm is the largest singular value: $\|W^{(k)}\| = \sigma_{\max}(W^{(k)})$. So each backward hop multiplies the norm by at most $\gamma \cdot \sigma_{\max}$.

Step 3 — collect the exponent. With every layer at roughly the same scale, the product of $L-l$ identical factors is that factor raised to the power $L-l$. Only the base $\gamma \cdot \sigma_{\max}$ matters, and the depth appears only as an exponent — which is why the effect is exponential in depth rather than linear. $\square$

Reading the condition. Three regimes. If $\gamma\sigma_{\max} < 1$, the blame decays geometrically with depth: by the time it reaches the early layers it is numerically zero, so those layers barely update and barely learn, and the network trains only its last few layers. This is the *vanishing gradient problem*. If $\gamma\sigma_{\max} > 1$, the blame grows geometrically and training destabilises. Only $\gamma\sigma_{\max} \approx 1$ trains properly, and the entire design space of activations and initializations is best understood as an attempt to sit on that knife edge.

The sigmoid case, with numbers. For the sigmoid, $\sigma'(z) = \sigma(z)(1 - \sigma(z))$. Writing $s = \sigma(z) \in (0, 1)$ and maximizing $g(s) = s - s^2$ gives $g'(s) = 1 - 2s = 0 \Rightarrow s^* = \frac{1}{2}$, with $g''(s) = -2 < 0$ confirming a global maximum. Hence $\gamma = \frac{1}{4}$ exactly — proven, not asserted, and true at the function’s single best point.

Now propagate the consequence. Holding $\gamma\sigma_{\max} \approx 1$ would require $\sigma_{\max} \approx 4$ in every layer, which means enormous weights; but large weights push z far from the origin, and that is exactly where $|\sigma'|$ collapses far below $\frac{1}{4}$. The two requirements are in direct conflict, so sigmoid deep networks are squeezed between vanishing and saturation with no escape. Without the escape, an L -layer stack scales the signal by at most $(\frac{1}{4})^L$: about 10^{-3} at $L = 5$, about 10^{-6} at $L = 10$, about 10^{-15} at $L = 25$. This is not “might vanish” — it cannot avoid shrinking by at least a million-fold across ten layers, before the weights are even consulted. For tanh the same argument gives $\gamma = 1$ at the origin, which is strictly better and is why tanh was preferred for hidden layers before ReLU. For ReLU, $\gamma = 1$ over the entire positive axis, and the condition reduces to $\sigma_{\max} \approx 1$ — achievable by initialization alone, which is Section 9.

The exploding half is the easier half. The two failure modes are not symmetric, in two respects. First in diagnosis: vanishing is insidious, since training simply underperforms and silently converges to something mediocre, whereas exploding announces itself as a loss spike followed by NaN. Second in remedy: exploding has a blunt, effective fix in global-norm clipping, $g \leftarrow g \cdot \min(1, c/\|g\|)$, which shortens the step while preserving its direction — prefer it to clipping each coordinate independently, which distorts the direction. Vanishing admits no mirror-image fix. One cannot simply scale a vanished gradient back up, because a signal lost to floating-point underflow has had its information destroyed, not merely attenuated; there is nothing left to amplify. That asymmetry is why the architectural literature is overwhelmingly about the vanishing side.

9 He initialization, from a variance constraint

Proposition 8.1 shows the activation supplies only half of the product $\gamma\sigma_{\max}$; the weight scale supplies the other half. Initialization is how that scale is set before training begins, and getting it wrong costs the network before the first step is taken — a good activation with a bad initialization vanishes just as thoroughly as a bad activation.

The design goal is a conservation law: choose $\text{Var}(W)$ so that the variance of the activations neither grows nor decays as signal passes through a layer. If it is preserved at every layer, it is preserved through the whole network at any depth.

Proposition 9.1. *For a ReLU network with fan-in n , i.i.d. zero-mean weights independent of the inputs, activation variance is preserved layer to layer when $\text{Var}(W) = 2/n$.*

Proof. We proceed in three steps.

Step 1 — variance of one pre-activation. With $z_i^{(l)} = \sum_{j=1}^n W_{ij} a_j^{(l-1)}$ a sum of n independent zero-mean terms, variances add, and for a product of independent factors with $\mathbb{E}[W] = 0$ we have $\text{Var}(Wa) = \mathbb{E}[W^2] \mathbb{E}[a^2] = \text{Var}(W) \mathbb{E}[a^2]$. Hence

$$\text{Var}(z^{(l)}) = n \text{Var}(W) \mathbb{E}\left[\left(a^{(l-1)}\right)^2\right]. \quad (36)$$

Note carefully that it is the second moment $\mathbb{E}[a^2]$ that appears, not $\text{Var}(a)$. The two coincide only for zero-mean a , and ReLU guarantees a nonzero mean since its output is never negative. Conflating them here is the single easiest way to derive the wrong constant.

Step 2 — the ReLU-specific step. This is the crux. If z is symmetric about 0, then z^2 is distributed identically on the two halves of the axis, so each half contributes equally to $\mathbb{E}[z^2]$. ReLU zeroes one of those halves outright. Exactly half the second moment therefore survives:

$$\mathbb{E}[a^2] = \mathbb{E}[\max(0, z)^2] = \frac{1}{2} \mathbb{E}[z^2] = \frac{1}{2} \text{Var}(z), \quad (37)$$

the last equality using $\mathbb{E}[z] = 0$. This factor of $\frac{1}{2}$ is the entire difference between He and Xavier initialization.

Step 3 — impose preservation. Substituting Step 2 into Step 1, $\text{Var}(z^{(l)}) = n \text{Var}(W) \cdot \frac{1}{2} \cdot \text{Var}(z^{(l-1)})$. Preservation means the multiplier is one: $n \text{Var}(W) \cdot \frac{1}{2} = 1$, i.e. $\text{Var}(W) = 2/n$. $\square$

Reading the result. He initialization is $W \sim \mathcal{N}(0, 2/n_{\text{in}})$, and the 2 has a precise meaning: it compensates for the half of the signal that ReLU discards. Xavier/Glorot instead targets $2/(n_{\text{in}} + n_{\text{out}})$, which compromises between preserving the forward signal (which wants $1/n_{\text{in}}$) and the backward one (which wants $1/n_{\text{out}}$), and which assumes a roughly linear, symmetric activation such as tanh that passes the full second moment. The rule of thumb — He with ReLU and its variants, Xavier with tanh or sigmoid — is not a matter of taste; it is Step 2 being present or absent.

Getting the factor of two wrong is not cosmetic. A per-layer multiplier of $\frac{1}{2}$ on the variance compounds as $(1/2)^{L/2}$ in the standard deviation, which over 25 layers costs several orders of magnitude — a network that is, in effect, untrained in its early layers purely because of a constant.

Remark 9.2 (Why not initialize to zero). Setting $\text{Var}(W) = 0$ gives $\text{Var}(z) = 0$, so every unit in a layer outputs the same value — but the fatal problem is on the backward pass. By Proposition 3.1, identical incoming weights mean every unit in the layer receives an identical δ , and by Proposition 4.1 identical δ with identical inputs means identical weight updates. The units therefore remain exact clones of one another forever: a 512-unit layer is permanently a 1-unit layer, no matter how long it trains. Any constant initialization, not just zero, has the same fate. This is the *symmetry-breaking* problem, and random initialization is what breaks it; Section 9 then chooses the scale once it is broken. Biases may safely start at zero, since the weights have already broken the symmetry.

10 The residual block

Every fix considered so far works *inside* the product of (21): ReLU improves the D factors, initialization improves the W factors, normalization keeps z in the region where D is healthy. The residual connection does something categorically different — it changes the product so that one term in it cannot be small.

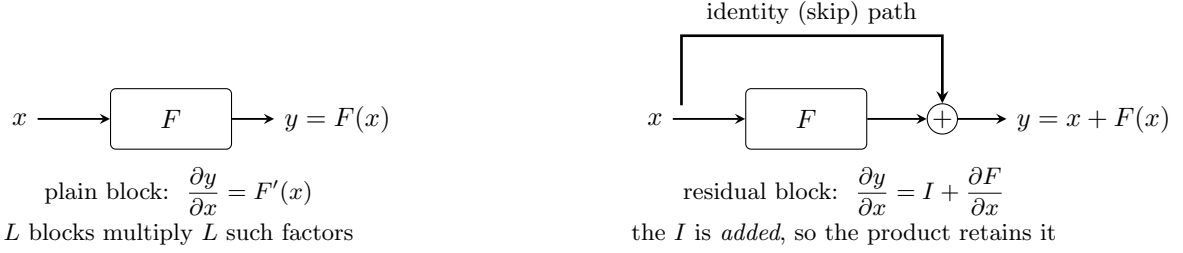


Figure 3: One symbol of difference. The plain block’s backward pass multiplies by F' and dies when $F' \approx 0$; the residual block multiplies by $I + \partial F / \partial x$, and by Proposition 10.1 the expansion of $\prod_k (I + J_k)$ over L blocks always contains a standalone I — a path of derivative exactly 1 from the loss to any layer. That term is the one thing the vanishing product of Section 8 cannot touch.

Proposition 10.1. For blocks $x_{k+1} = x_k + F(x_k)$ with $J_k := \partial F / \partial x_k$,

$$\frac{\partial \mathcal{L}}{\partial x_l} = \frac{\partial \mathcal{L}}{\partial x_L} \prod_{k=l}^{L-1} (I + J_k) = \frac{\partial \mathcal{L}}{\partial x_L} \left(I + \sum_k J_k + \sum_{k < j} J_k J_j + \cdots \right). \quad (38)$$

Proof. Differentiating a single block gives $\partial x_{k+1} / \partial x_k = I + J_k$: the identity path contributes I , and crucially that I is *added* to the block’s Jacobian rather than multiplied by it. Chaining through L blocks as in (21) gives the product. Expanding a product of binomials enumerates every way of choosing, from each factor, either the I or the J_k — that is, every subset of $\{J_l, \dots, J_{L-1}\}$. The subset of size zero contributes exactly I . $\square$

Reading the guarantee carefully. Compare with the plain-network product $\prod_k D^{(k)}(W^{(k)})^\top$ of (21), in which nothing prevents a factor from being tiny and therefore nothing prevents the whole product from being tiny. In a plain network, if the J ’s are small the gradient is gone, because the product of small things is smaller still. In a residual network the worst case is completely different: if every $J_k \rightarrow 0$, the expansion does not tend to zero, it tends to I . The deep network degenerates gracefully into a shallower one rather than into silence, and the gradient can always get home.

The same expansion answers a second question that is often asked separately — *why is a residual easier to learn?* If the optimal map for a block is close to the identity, then F needs to learn only ≈ 0 , which is trivial, whereas a plain block would have to learn the identity map exactly, which is surprisingly hard for a randomly initialised layer. The reparameterisation moves the “do nothing” solution from somewhere awkward in weight space to the origin.

The general principle. Worth extracting and stating cold: *additive paths preserve gradients; multiplicative paths destroy them.* The LSTM cell state (1997) and the ResNet skip connection (2015) are two instances of the same fix, eighteen years apart, and once the principle is named the resemblance is obvious rather than surprising.

The principle also explains a detail that otherwise looks arbitrary, namely where normalization is placed relative to the skip. Post-norm, $x \leftarrow \text{LayerNorm}(x + F(x))$, puts a transformation *on* the identity path, so the clean I of Proposition 10.1 is no longer clean and deep stacks need careful warmup to train at all. Pre-norm, $x \leftarrow x + F(\text{LayerNorm}(x))$, normalizes only the branch and leaves the identity path unobstructed from the loss all the way to the embeddings, which trains far more robustly at depth. Moving one operation across an addition is the difference, and the reason is entirely contained in whether the standalone I survives.

11 Verifying an implementation

A derived gradient and an implemented gradient are different things, and the gap between them is where most of the debugging time goes. Finite differences, useless for training, is the right tool for testing precisely because it shares no code with the backward pass.

Use the two-sided form (1) rather than the one-sided $(\mathcal{L}(\theta + \epsilon) - \mathcal{L}(\theta))/\epsilon$: the two-sided estimate has error $O(\epsilon^2)$ against the one-sided $O(\epsilon)$, for twice the cost. Compare against the analytic gradient with a relative rather than absolute criterion,

$$\frac{|g_{\text{analytic}} - g_{\text{numeric}}|}{\max(|g_{\text{analytic}}|, |g_{\text{numeric}}|, \epsilon)}, \quad (39)$$

since an absolute difference of 10^{-6} means something quite different for a gradient of magnitude 10^{-8} than for one of magnitude 10^2 . In double precision, values below about 10^{-7} indicate agreement and values above about 10^{-3} indicate a genuine bug.

Four practical cautions. Run the check in double precision, since single precision leaves too little headroom between truncation error and rounding error for the comparison to mean anything. Check a small network with few parameters — the point is correctness, not scale. Disable any stochastic component (dropout, or anything else that resamples per forward pass) or fix its random seed, since $\mathcal{L}(\theta + \epsilon)$ and $\mathcal{L}(\theta - \epsilon)$ must differ only by the perturbation. And be aware of kinks: at a non-differentiable point such as ReLU's origin the two-sided difference straddles the kink and will disagree with the subgradient the framework returns, which is a limitation of the test rather than a bug in the code.

12 Summary

$\delta^{(L)} = \nabla_{a^{(L)}} \mathcal{L} \odot f'(z^{(L)}) \quad (\text{base case; } = \hat{y} - y \text{ for softmax-cross-entropy})$
$\delta^{(l)} = \left(\left(W^{(l+1)} \right)^\top \delta^{(l+1)} \right) \odot f'(z^{(l)}) \quad (\text{recursion})$
$\frac{\partial \mathcal{L}}{\partial W^{(l)}} = \delta^{(l)} \left(a^{(l-1)} \right)^\top, \quad \frac{\partial \mathcal{L}}{\partial b^{(l)}} = \delta^{(l)} \quad (\text{parameter gradients})$

Three lines, and they are the same three lines read in two directions. Read forward, as an algorithm: reusing each δ rather than recomputing it turns an $O(P^2)$ job into an $O(P)$ one, which is dynamic programming on the computation graph and is what made training deep networks feasible. Read again, as a warning: the recursion is multiplicative, so the blame reaching layer l is scaled by roughly $(\gamma \sigma_{\max})^{L-l}$ — below one it vanishes exponentially, above one it explodes.

The rest of the subject follows from asking, of any proposed technique, which part of that product it protects. Sigmoid's $\gamma = \frac{1}{4}$ makes vanishing unavoidable, and ReLU's $\gamma = 1$ removes the shrinkage from the D factors; He initialization's $\text{Var}(W) = 2/n$ puts the W factors at the right scale; normalization keeps z in the region where D stays healthy; gradient clipping tames the exploding half; and the residual block's $I + J_k$ removes the dependence on the product altogether. One equation, and most of the architectural history of deep learning as its consequences.